GISviz-checkpoint

April 13, 2026

[1]:
```
from pathlib import Path
import os
import json
import math
import glob

import numpy as np
import pandas as pd

try:
    import geopandas as gpd
except ImportError:
    gpd = None

try:
    import rasterio
    from rasterio.transform import from_origin
except ImportError:
    rasterio = None

try:
    from scipy.interpolate import griddata
except ImportError:
    griddata = None
```

[2]:
```
# --- main project folders ---
ROOT = Path("/home/jovyan")
RAW_DIR = ROOT / "raw_data"
DEPTH_GRID_DIR = RAW_DIR / "north_slope_depth_grids"

# where your parquet layers are / will be
PARQUET_DIRS = [
    ROOT / "parquets",
    ROOT / "processed_data",
    ROOT / "raw_data",
]

# output folder for rasters made from XYZ surfaces
```

```
RASTER_OUT_DIR = DEPTH_GRID_DIR / "rasters_from_xyz"
RASTER_OUT_DIR.mkdir(parents=True, exist_ok=True)

# manifest outputs
MANIFEST_DIR = ROOT / "project_manifests"
MANIFEST_DIR.mkdir(parents=True, exist_ok=True)

TXT_MANIFEST = MANIFEST_DIR / "north_slope_file_manifest.txt"
CSV_MANIFEST = MANIFEST_DIR / "north_slope_file_manifest.csv"

print("DEPTH_GRID_DIR:", DEPTH_GRID_DIR)
print("RASTER_OUT_DIR:", RASTER_OUT_DIR)
print("MANIFEST_DIR:", MANIFEST_DIR)
```

```
DEPTH_GRID_DIR: /home/jovyan/raw_data/north_slope_depth_grids
RASTER_OUT_DIR: /home/jovyan/raw_data/north_slope_depth_grids/rasters_from_xyz
MANIFEST_DIR: /home/jovyan/project_manifests
```

[ ]:
```
def scan_files(base_dirs, extensions=None):
    rows = []

    for base in base_dirs:
        if not base.exists():
            continue

        for path in base.rglob("*"):
            if path.is_file():
                if extensions:
                    if path.suffix.lower() not in extensions:
                        continue

                rows.append({
                    "name": path.name,
                    "suffix": path.suffix.lower(),
                    "parent": str(path.parent),
                    "full_path": str(path.resolve()),
                    "size_mb": round(path.stat().st_size / (1024**2), 3),
                    "modified": pd.to_datetime(path.stat().st_mtime, unit="s"),
                })

    df = pd.DataFrame(rows).sort_values(["suffix", "name", "modified"]).
 ↪reset_index(drop=True)
    return df

base_dirs_to_scan = [
    ROOT,
]
```

```
manifest_df = scan_files(
    base_dirs_to_scan,
    extensions={".parquet", ".xyz", ".tif", ".tiff", ".ipynb", ".csv", ".txt",␣
 ↪".shp", ".geojson"}
)

manifest_df.head(20)
print(f"Found {len(manifest_df)} matching files.")
```

[ ]:
```
manifest_df.to_csv(CSV_MANIFEST, index=False)

with open(TXT_MANIFEST, "w", encoding="utf-8") as f:
    f.write("NORTH SLOPE PROJECT FILE MANIFEST\n")
    f.write("=" * 80 + "\n\n")

    for _, row in manifest_df.iterrows():
        f.write(f"NAME:     {row['name']}\n")
        f.write(f"TYPE:     {row['suffix']}\n")
        f.write(f"SIZE_MB:  {row['size_mb']}\n")
        f.write(f"MODIFIED: {row['modified']}\n")
        f.write(f"PARENT:   {row['parent']}\n")
        f.write(f"PATH:     {row['full_path']}\n")
        f.write("-" * 80 + "\n")

print("Saved:")
print(TXT_MANIFEST)
print(CSV_MANIFEST)
```

[ ]:
```
parquet_df = manifest_df[manifest_df["suffix"] == ".parquet"].copy()
display(parquet_df[["name", "parent", "full_path", "size_mb"]].
 ↪sort_values("name"))
print(f"Total parquet files: {len(parquet_df)}")
```

[ ]:
```
loaded_layers = {}

for _, row in parquet_df.iterrows():
    path = Path(row["full_path"])
    key = path.stem.replace(" ", "_").replace("-", "_")

    try:
        df = pd.read_parquet(path)
        loaded_layers[key] = df
        print(f"Loaded: {key:40s} | shape={df.shape}")
    except Exception as e:
        print(f"FAILED: {path.name} -> {e}")
```

```
print(f"\nLoaded {len(loaded_layers)} parquet datasets.")
```

[ ]:
```
for name, df in loaded_layers.items():
    print("\n" + "=" * 80)
    print(name)
    print(df.head(3))
    print(df.columns.tolist())
```

[ ]:
```
from pathlib import Path

ROOT = Path("/home/jovyan/notebooks")
RAW_DIR = ROOT / "raw_data"
DEPTH_GRID_DIR = RAW_DIR / "north_slope_depth_grids"

print("DEPTH_GRID_DIR =", DEPTH_GRID_DIR)
print("Exists?        =", DEPTH_GRID_DIR.exists())
print("Is dir?        =", DEPTH_GRID_DIR.is_dir())

xyz_files = sorted(DEPTH_GRID_DIR.glob("*.XYZ")) + sorted(DEPTH_GRID_DIR.
 ↪glob("*.xyz"))

print("\nXYZ files found:")
for p in xyz_files:
    print(" -", p.name)

print(f"\nTotal XYZ files: {len(xyz_files)}")
```

[ ]:
```
def read_xyz_file(path):
    rows = []

    with open(path, "r", encoding="utf-8", errors="ignore") as f:
        for line in f:
            line = line.strip()

            # skip headers and separators
            if (
                not line
                or line.startswith("/")
                or "X" in line and "Y" in line and "Z" in line
                or "----" in line
            ):
                continue

            parts = line.split()

            # expect at least 5 columns
            if len(parts) < 5:
```

```
                continue

            try:
                x = float(parts[0])      # projected X (ignore)
                y = float(parts[1])      # projected Y (ignore)
                z = float(parts[2])      # depth
                lon = float(parts[3])    # longitude
                lat = float(parts[4])    # latitude

                rows.append((lon, lat, z))

            except ValueError:
                continue

    if not rows:
        raise ValueError(f"No valid rows found in {path.name}")

    df = pd.DataFrame(rows, columns=["lon", "lat", "z"])
    return df
```

[ ]:
```
test_xyz = DEPTH_GRID_DIR / "NSshublik.XYZ"

with open(test_xyz, "r", encoding="utf-8", errors="ignore") as f:
    for i in range(15):
        line = f.readline()
        print(f"{i+1:02d}: {repr(line)}")
```

[ ]:
```
test_xyz = DEPTH_GRID_DIR / "NSshublik.XYZ"
print("Testing:", test_xyz)
print("Exists? ", test_xyz.exists())

test_df = read_xyz_file(test_xyz)

print(test_df.head())
print(test_df.describe())
print("Rows:", len(test_df))
```

[ ]:
```
def estimate_spacing(values):
    vals = np.sort(np.unique(values))
    diffs = np.diff(vals)
    diffs = diffs[diffs > 0]
    if len(diffs) == 0:
        return None
    return float(pd.Series(diffs).mode().iloc[0])

dx = estimate_spacing(test_df["lon"].values)
dy = estimate_spacing(test_df["lat"].values)
```

```
print("Estimated dx:", dx)
print("Estimated dy:", dy)
```

[27]:
```
import rasterio
from rasterio.transform import from_origin

def xyz_to_raster_lonlat(df, out_tif, crs="EPSG:4326", nodata=-9999.0):
    # unique sorted coordinates
    lon_vals = np.sort(df["lon"].unique())
    lat_vals = np.sort(df["lat"].unique())

    dx = estimate_spacing(lon_vals)
    dy = estimate_spacing(lat_vals)

    if dx is None or dy is None:
        raise ValueError("Could not determine lon/lat spacing.")

    # build grid
    grid = df.pivot_table(index="lat", columns="lon", values="z",␣
 ↪aggfunc="mean")

    # force full coordinate coverage
    grid = grid.reindex(index=lat_vals, columns=lon_vals)

    arr = grid.to_numpy(dtype="float32")

    # replace nan with nodata
    arr = np.where(np.isnan(arr), nodata, arr)

    # flip so north is at top
    arr = np.flipud(arr)

    # upper-left corner transform
    west = lon_vals.min() - dx / 2
    north = lat_vals.max() + dy / 2
    transform = from_origin(west, north, dx, dy)

    with rasterio.open(
        out_tif,
        "w",
        driver="GTiff",
        height=arr.shape[0],
        width=arr.shape[1],
        count=1,
        dtype="float32",
        crs=crs,
```

```
        transform=transform,
        nodata=nodata,
    ) as dst:
        dst.write(arr, 1)

    return out_tif
```

[28]:
```
from pathlib import Path

RASTER_OUT_DIR = DEPTH_GRID_DIR / "rasters_from_xyz"
RASTER_OUT_DIR.mkdir(parents=True, exist_ok=True)

print("Raster output folder:", RASTER_OUT_DIR)
```

```
Raster output folder:
/home/jovyan/notebooks/raw_data/north_slope_depth_grids/rasters_from_xyz
```

[ ]:
```
xyz_files = sorted(DEPTH_GRID_DIR.glob("*.XYZ")) + sorted(DEPTH_GRID_DIR.
 ↪glob("*.xyz"))

raster_outputs = []

for xyz_path in xyz_files:
    try:
        df = read_xyz_file(xyz_path)
        out_tif = RASTER_OUT_DIR / f"{xyz_path.stem}.tif"
        xyz_to_raster_lonlat(df, out_tif)
        raster_outputs.append(out_tif)
        print(f"OK   -> {xyz_path.name} -> {out_tif.name}")
    except Exception as e:
        print(f"FAIL -> {xyz_path.name}: {e}")

print(f"\nCreated {len(raster_outputs)} rasters.")
```

```
/tmp/ipykernel_1347/1769764110.py:16: PerformanceWarning: The following
operation may generate 4079985728 cells in the resulting pandas object.
  grid = df.pivot_table(index="lat", columns="lon", values="z", aggfunc="mean")
```

[ ]:
```

```